RICHFIELD
richfield.ac.za

Name and Surname: Tashen Nair

Student ITS No:402204898

Qualification: BSCIT **Year of Study: 3rd** **Semester: 2nd**

Assignment due date: **Date submitted: 30/09/2026**

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document. Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents

Introduction

Main Body (with relevant subheadings)

Conclusion

References

Submit the assignment in PDF format on Moodle.

Use the specified cover page provided.

Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature: nair **Date:**

Richfield Graduate Institute of Technology (Pty) Ltd is registered with the Department of Higher Education & Training as a Private Higher Education Institution under the Higher Education Act, 1997, Registration Certificate No. 2000/HE07/008.

Table of Contents

Introduction 2

Screen flow/wireframe diagram showing how the screens connect, and a simple data model/ER diagram of the database table(s)/collection(s). 2

Screenshots of the Smart Pantry Manager screens: 7

Key Code Snippets (4): 11

Challenges and Solutions: 17

Conclusion and Reflection: 18

Reference List: 19

Introduction

Food waste is a major problem in many households. People often buy new groceries while forgotten ingredients sit in the back of their cupboards and spoil. Most recipe applications focus on planned meals that require buying specific items from the store, which encourages unnecessary shopping trips and leaves partial ingredients to waste. 3.2 Target Audience & Solution The Smart Pantry Manager is designed for students, busy workers, and home cooks who want to save money and reduce food waste by using what they already have. The app tracks inventory directly on the user's phone (ingredient name, quantity, unit, and expiration date). Its main feature is a strict recipe-matching algorithm that only suggests meals the user can cook right now with their current stock—requiring zero shopping trips.

Screen flow/wireframe diagram showing how the screens connect, and a simple data model/ER diagram of the database table(s)/collection(s).

Screen Flow Diagram The application follows a modular navigation architecture using explicit Android Intents to pass data between screens:

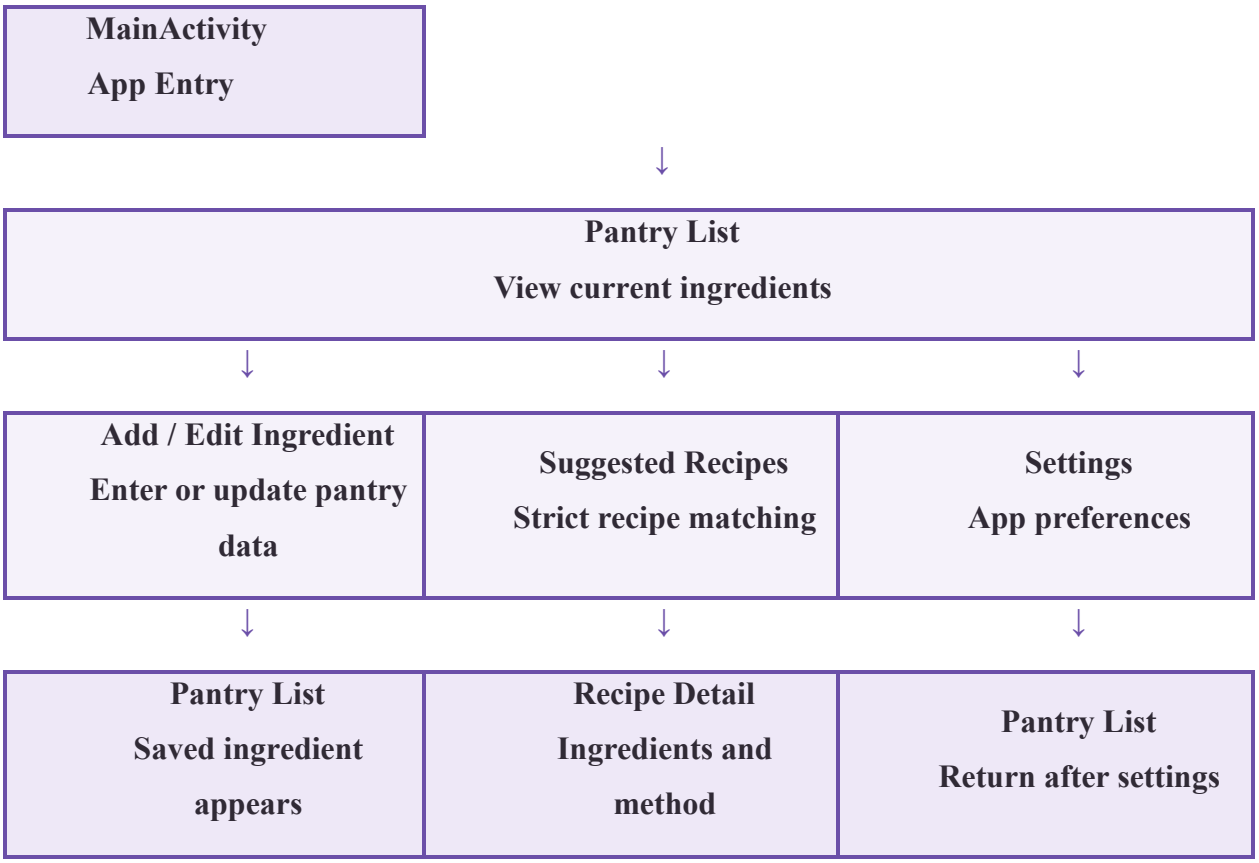
Smart Pantry Manager

Screen Flow / Wireframe and Database ER Model

Editable Word diagrams for the Written Report

1. Screen Flow / Wireframe

The boxes below are Word tables, not images, so they can be edited directly in Microsoft Word.



Navigation: Add/Edit returns to Pantry after saving. Suggested Recipes opens Recipe Detail when a recipe is selected. Settings returns to Pantry.

Screen contents

Screen	Main elements
Pantry List	Ingredient list, Add Ingredient, Suggested Recipes, Settings
Add/Edit Ingredient	Ingredient name, quantity, unit, expiry date, Save, Back
Suggested Recipes	Recipes passing strict matching, or a no-match message
Recipe Detail	Recipe name, description, ingredients, preparation method
Settings	Expiry reminders, Dark mode, Delete confirmations, Simplify quantities

2. Simple Data Model / ER Diagram

The app uses a local SQLite database with two main tables: ingredients and recipes.

INGREDIENTS	LOGICAL MATCHING	RECIPES
PK id : INTEGER name : TEXT quantity : REAL unit : TEXT expiryDate : TEXT	Pantry ingredients | v Strict matching | v Suggested recipes	PK id : INTEGER name : TEXT description : TEXT ingredients : TEXT instructions : TEXT

Important: There is no direct foreign-key relationship between these two tables. The application compares recipe requirements with pantry ingredients in Java. A recipe is suggested only when all required ingredients and quantities are available.

Database table details

Table	Field	Type	Purpose
ingredients	**id**	INTEGER	Primary key
ingredients	**name**	TEXT	Ingredient name
ingredients	**quantity**	REAL	Amount available
ingredients	**unit**	TEXT	Measurement unit
ingredients	**expiryDate**	TEXT	Optional expiry date
recipes	**id**	INTEGER	Primary key
recipes	**name**	TEXT	Recipe name

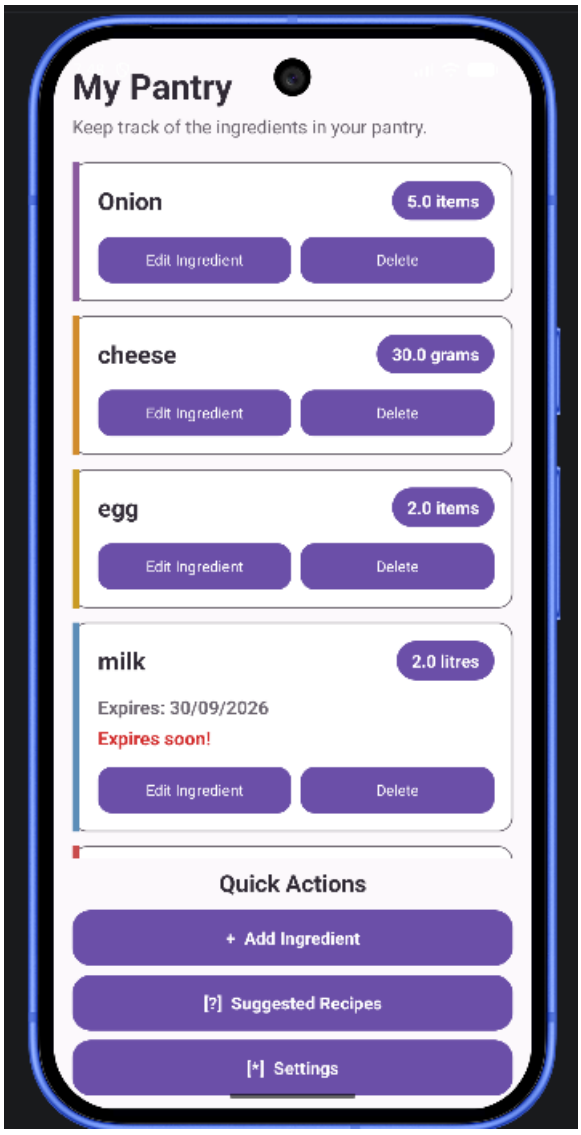
recipes	**description**	TEXT	Short recipe description
recipes	**ingredients**	TEXT	Required ingredients and quantities
recipes	**instructions**	TEXT	Preparation method

Assignment alignment: The structure supports pantry CRUD, the pre-loaded recipe collection, persistent SQLite storage, and the strict-matching requirement in the assignment.

Data Model / ER Diagram (SQLite) The application uses local persistent storage with an SQLite database managed by DatabaseHelper.java.

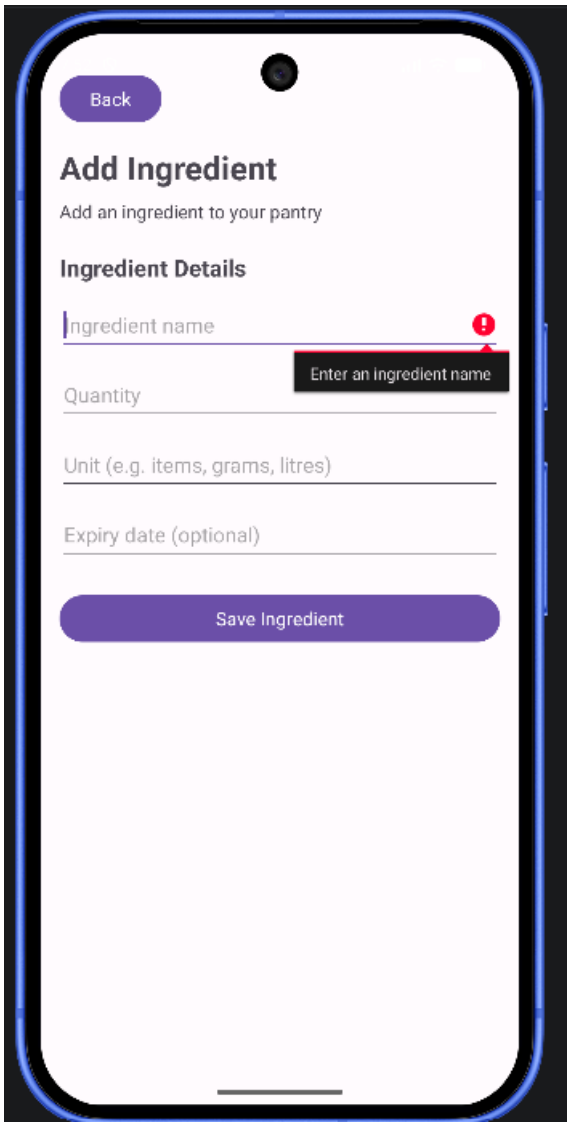
Screenshots of the Smart Pantry Manager screens:

My Pantry Screen:



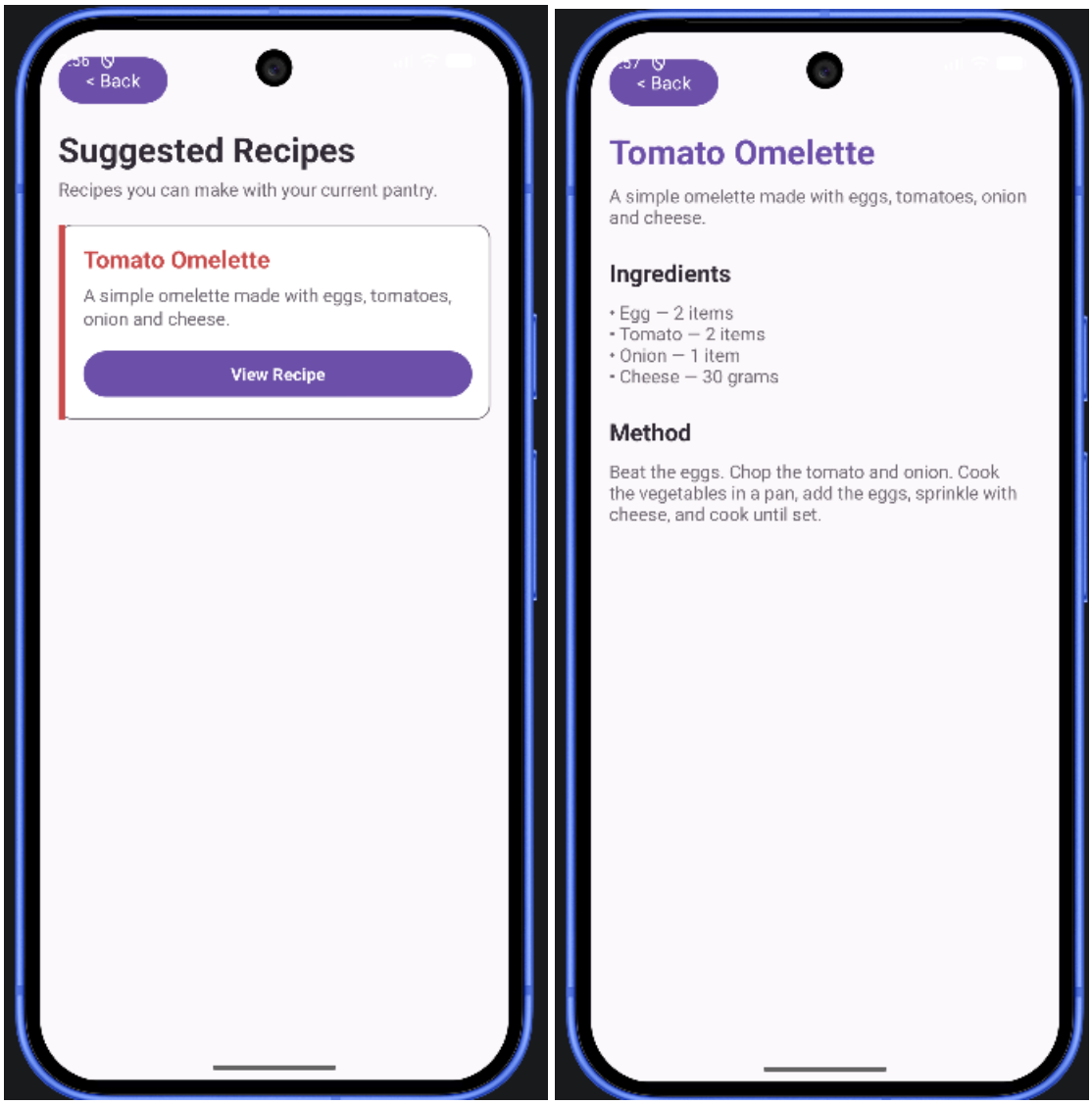
This is the screen of my pantry. I added a few ingredients and each ingredients are color coded e.g. tomato = red, cheese = yellow

Add Ingredient screen:



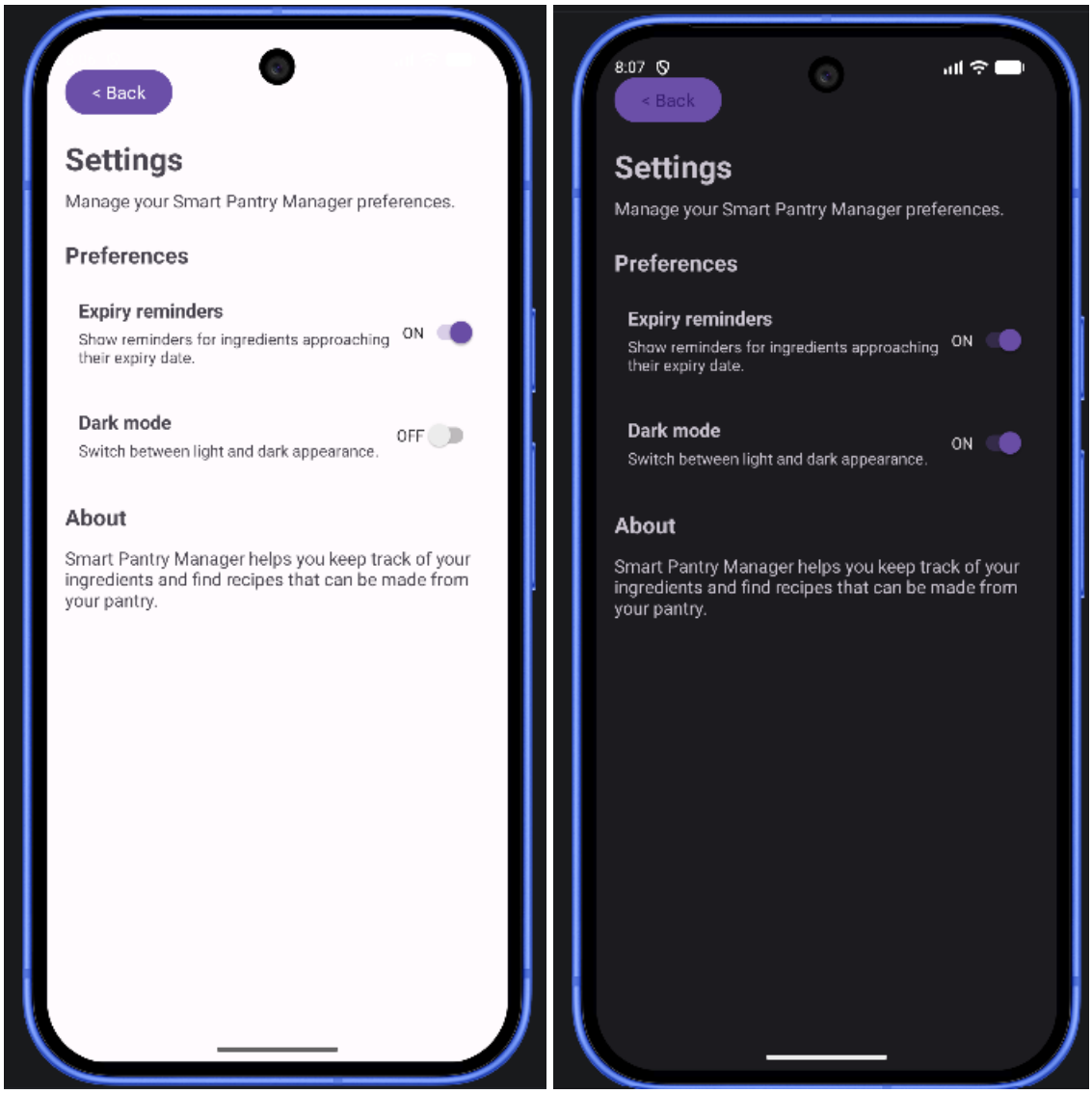
When you click “Add ingredient” then you will be brought to this screen. If you click “Save ingredient” but you did not add an ingredient name then an error will appear and tell you what is required, in this screenshot I demonstrated what happens if you click “Save ingredient” but don’t add an ingredient name, this also happens when you don’t add quantity and unit, expiry date Is optional so you can click “save ingredient” without it.

Suggested Recipes Screen:



When you click "Suggested recipes" you will see the first screen with the title "Suggested Recipes." If you don't have enough ingredients to make a suggested recipe then there wont be any suggested recipes but instead a message will pop up: "No recipes match your pantry as yet." When you click "View Recipe" then you will see more information about the suggested recipe. My suggested recipe was a **Tomato Omelette,** and when I clicked "View recipe" then it shows me the ingredients from my pantry and the method to make the **Tomato Omelette.**

Settings Screen:



The settings screen is where the user can customize their Smart Pantry Manager, in this version there is only 2 preferences implemented, adding a decimal conversion setting and a zoom feature setting couldn't be added to this version due to limited time. The first preference setting is "Expiry reminders", if the user click this off then they wont receive warnings about an ingredient expiring. My first screenshot which was the My Pantry screen had the ingredient "milk" and it had an expiring warning. The second preference setting is "Dark mode". A lot of people prefer using dark mode for multiple applications e.g. Microsoft Word, so the Smart Pantry Manager should be no exception.

Key Code Snippets (4):

1. Strict Recipe-Matching Logic

```
private boolean canMakeRecipe(
    Recipe recipe,
    List<Ingredient> pantryIngredients) {

String[] requiredIngredients =
    recipe.getIngredients().split(",");

for (String required : requiredIngredients) {

String[] parts = required.split(":", 3);

if (parts.length != 3) {
    return false;
}

String requiredName = normalizeName(parts[0]);
double requiredQuantity;

try {
    requiredQuantity = Double.parseDouble(parts[1]);
} catch (NumberFormatException e) {
    return false;
```

```
}

String requiredUnit = normalizeUnit(parts[2]);

double requiredBaseQuantity =
        convertToBaseQuantity(
                requiredQuantity,
                requiredUnit
        );

double pantryBaseQuantity = 0;

for (Ingredient pantryIngredient : pantryIngredients) {

    String pantryName =
            normalizeName(pantryIngredient.getName());

    String pantryUnit =
            normalizeUnit(pantryIngredient.getUnit());

    if (pantryName.equals(requiredName)
            && areCompatibleUnits(
            pantryUnit,
            requiredUnit)) {
```

```
            double pantryQuantity =
                    convertToBaseQuantity(
                            pantryIngredient.getQuantity(),
                            pantryUnit
                    );

            pantryBaseQuantity += pantryQuantity;
        }
    }

    if (pantryBaseQuantity < requiredBaseQuantity) {
        return false;
    }
  }

  return true;
}
```

Explanation:

This is the main business logic of the application. It checks every ingredient required by a recipe against the ingredients currently stored in the user's pantry. The recipe is only suggested when every required ingredient is available in a sufficient quantity. The code also normalises ingredient names and units so that simple differences such as "tomato" and "tomatoes", or kilograms and grams, can be handled. This was written this way because the assignment specifically requires strict recipe matching rather than partial matches.

2 Adding and Updating Pantry Ingredients

```
ContentValues values = new ContentValues();


values.put("name", name);

values.put("quantity", quantity);

values.put("unit", unit);

values.put("expiryDate", expiryDate);


long result;


if (ingredientId == -1) {
   result = db.insert("ingredients", null, values);
} else {
   result = db.update(
         "ingredients",
         values,
         "id = ?",
         new String[]{String.valueOf(ingredientId)}
   );
}
```

Explanation:

This code handles both adding a new ingredient and updating an existing ingredient. ContentValues is used to place the user's input into the SQLite database. The ingredientId determines whether the operation should insert a new record or update an existing one. Combining these operations reduced duplicated code and allowed the same screen to be used for both adding and editing ingredients.

3. RecyclerView and Custom Adapter

```
recyclerViewIngredients.setLayoutManager(
        new LinearLayoutManager(this)
);

ingredientAdapter =
        new IngredientAdapter(ingredientList);

recyclerViewIngredients.setAdapter(
        ingredientAdapter
);
```

Explanation:

This code connects the pantry data to the RecyclerView. The custom IngredientAdapter controls how each ingredient is displayed, including its name, quantity, expiry information, edit button and delete button. RecyclerView was used because it is designed for displaying dynamic lists of data efficiently, with an Adapter responsible for connecting the data to the individual list items. (Android Developers, 2026a).

4. Navigation Using an Intent

```
Intent intent = new Intent(
        v.getContext(),
        AddIngredientActivity.class
);

intent.putExtra(
```

```
        "ingredient_id",
        ingredient.getId()
);

intent.putExtra(
        "ingredient_name",
        ingredient.getName()
);

intent.putExtra(
        "ingredient_quantity",
        ingredient.getQuantity()
);

v.getContext().startActivity(intent);
```

Explanation:

This code opens the Add/Edit Ingredient screen from the Pantry screen. It also passes the selected ingredient's information to the next Activity so that the existing values can be displayed when the user edits an ingredient. Android uses Intents to start Activities and to pass small amounts of data between them. (Android Developers, 2026b).

Challenges and Solutions:

Challenge 1: Implementing strict recipe matching

One of the main challenges was making sure that the application did not display recipes when the user was missing an ingredient or did not have enough of an ingredient. A simple exact-name comparison was also not sufficient because users could enter singular or plural ingredient names, such as "tomato" and "tomatoes", or use different units such as grams and kilograms.

Solution:
I created a strict matching method that checks every ingredient required by a recipe. Ingredient names are normalised before comparison, and compatible units are converted to a common base unit. For example, kilograms are converted to grams and litres are converted to millilitres. The recipe is rejected as soon as one required ingredient is missing or the available quantity is insufficient.

Challenge 2: Managing add, edit and delete operations

Another challenge was making the pantry management functionality work correctly while keeping the displayed RecyclerView in sync with the SQLite database. In particular, deleting an ingredient initially did not remove it correctly from the displayed list.

Solution:
The delete operation was changed to remove the selected database record and then remove the corresponding item from the adapter list using its current adapter position. A confirmation dialog was also added so that an ingredient could not be accidentally deleted. The same database is used when the pantry is reopened, which allows the data to remain available after the application is closed.

Challenge 3: Android XML and layout errors

During development, some XML changes caused build errors. One example occurred when the Back button was written as < Back directly inside an XML attribute. The < character has a special meaning in XML and caused the resource compilation to fail.

Solution:
The character was replaced with the XML-safe form <, which displays the < character while keeping the XML valid. I also had to make sure that the IDs used in Java matched the IDs defined in the XML layouts. This helped me understand that Android Java code and XML layouts must work together and that changing an ID in one file can cause compilation errors in another file.

Conclusion and Reflection:

Developing the Smart Pantry Manager helped me understand how the different parts of an Android application work together. I gained practical experience with Java, Android Activities, XML layouts, Intents, RecyclerView, custom Adapters and SQLite database storage.

The most important concept I learned was how to turn a requirement into actual application logic. The strict recipe-matching requirement required more than simply checking whether an ingredient existed. I had to consider ingredient names, quantities and different measurement units before deciding whether a recipe could be suggested.

I also learned that developing an Android application requires continuous testing. Small changes to XML IDs, Java code or database logic can affect other parts of the application. Debugging these problems helped me become more comfortable with reading Android Studio error messages and tracing problems back to their source.

If I had more time, I would improve the application by adding more recipes and making the recipe ingredient data more structured instead of storing the ingredient list as text. I would also improve the user interface further and add additional validation and testing for unusual ingredient names, units and quantities. Another improvement would be to add automated tests for the strict recipe-matching logic so that different combinations of pantry ingredients and quantities could be tested more systematically.

Overall, the project gave me a better understanding of how a complete Android application is planned, implemented, tested and documented. It also showed me the importance of designing the database, user interface and application logic to work together rather than treating each part as a separate component.

Reference List:

Android Developers (2026a) ***Create dynamic lists with RecyclerView*****. Available at: https://developer.android.com/develop/ui/views/layout/recyclerview (Accessed: 30 September 2026).**

Android Developers (2026b) ***Intents and intent filters*****. Available at: https://developer.android.com/guide/components/intents-filters (Accessed: 30 September 2026).**

Android Developers (2026c) ***The activity lifecycle*****. Available at: https://developer.android.com/guide/components/activities/activity-lifecycle (Accessed: 30 September 2026).**

Android Developers (2026d) ***Customize a dynamic list*****. Available at: https://developer.android.com/develop/ui/views/layout/recyclerview-custom (Accessed: 30 September 2026).**